

PA1: From Bellman Updates to TD Control

Seher Kanwal · Student ID · Reinforcement Learning (Graduate) · October 2026

All graded numbers are those in results.json (spec settings: $\alpha = 0.5$, $\epsilon = 0.1$, 3,000 episodes, 400-step cap, seeds 0–4). Items marked [supp.] are supplementary: they come from extra_experiments.py, which reuses agent.epsilon_greedy and the same update rules (a check confirms it reproduces agent.sarsa / agent.q_learning bit for bit) and uses 20 seeds where stated. Reproduction commands, seeds and versions are in README.md.

1 Policy iteration vs value iteration

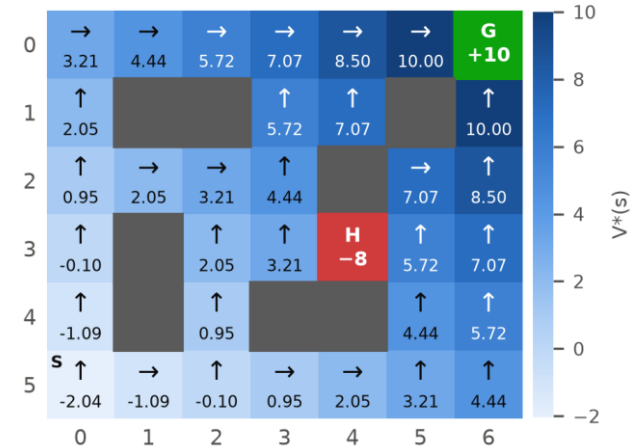


Figure 1. RoomsGridWorld: V^* (colour and value) and the greedy policy (arrows). PI and VI produce exactly this map.

Both methods reach the same optimal policy and values. $\max |V_{PI} - V_{VI}| = 0$ and every state gets the same action. The greedy path from S reaches G in 11 steps with undiscounted return 0 and $V^*(S) = -2.038$. At S, *up* and *right* are exactly tied ($Q = -2.038$ for both; each route is 11 steps), and the lowest-index rule selects *up*.

The counts differ because they count different things. A PI round contains a complete policy evaluation; a VI sweep is a single backup of every state. Counted in sweeps, PI needs 3,276 against 12 for VI, about 270 times more backups (Table 1).

	Policy iteration	Value iteration
Iterations	9 rounds	12 sweeps
Eval. sweeps per round	540 (rounds 1–6), 12 (7–9)	—
Total sweeps	3,276 + 9 improvements	12
$V^*(S)$, action at S	-2.038, up	-2.038, up

Table 1. DP summary; values identical to results.json.

Why rounds 1–6 cost 540 sweeps each. The initial all-up policy drives most states into a wall or the top border forever, so their value is $-1/(1-\gamma) = -20$. Under a fixed policy the change per sweep shrinks only by $\gamma = 0.95$, and $0.95^{k-1} < 10^{-12}$ gives exactly $k = 540$. Each improvement re-routes only the 4–7 states next to already-good ones, so some state keeps looping until round 6. From round 7 every path terminates, and evaluating a deterministic terminating policy is exact after 12 sweeps. Note that each evaluation starts from $V = 0$ (the initialisation convention). Warm-starting from the previous round's V would avoid recomputing the -20 of looping states from scratch, so the sweep total partly reflects this choice; the round count (9) does not, because each evaluation is run to the same fixed point.

Why VI needs only 12. The max lets correct values spread outward from G one cell per sweep: the largest change is exactly $10 \cdot 0.95^{k-1}$ (10, 9.5, 9.03, ..., 5.99). S is the farthest state, 11 steps away, so after 11 sweeps every value is exact and sweep 12 confirms $\Delta = 0$. PI takes a few large, expensive steps (policy improvement is a Newton-like jump); VI takes many cheap ones. On this small deterministic grid VI is far cheaper.

2 The paths learned by SARSA and Q-learning

Q-learning learns the shortest path along the cliff edge; SARSA learns a detour through the top rows (Figure 2). Q-learning's greedy path runs along row 3 on every seed: 11 steps, return -11.0 , **highest_row = 3.0**, identical to the DP optimum. SARSA's path climbs to row 0 (four seeds) or row 1 (one seed) before crossing: 15–17 steps, return -16.6 ± 0.8 , **highest_row = 0.2**. With 20 seeds [supp.] the picture is the same: Q-learning 3.0 on every seed, SARSA rows 0–1 only.

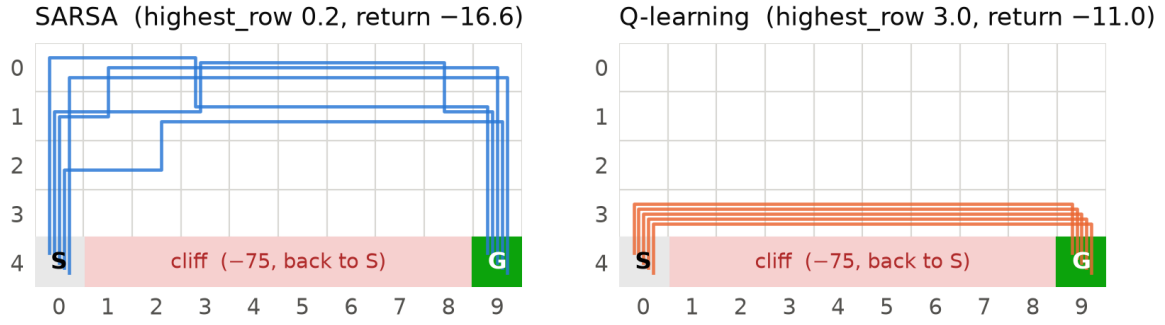
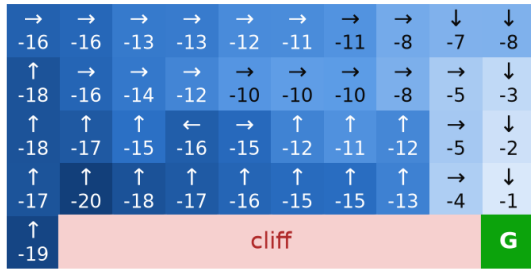


Figure 2. Greedy path of each seed (0–4), drawn slightly offset. Q-learning's path is identical on all seeds.

Why. Q-learning's target $r + \gamma \max_{a'} Q(s', a')$ assumes greedy behaviour from the next step on, so it estimates q^* , the value of the optimal policy, whatever exploration is actually performed (Bellman optimality). SARSA's target $r + \gamma Q(s', a')$ uses the action the ϵ -greedy policy actually takes, so it estimates q_π of the ϵ -greedy policy itself (Bellman expectation). On row 3 every step has a $0.1 \times \frac{1}{4} = 2.5\%$ chance of a random *down* move: -75 and a restart at S. SARSA's values price that risk in; Q-learning's do not.

Figure 3 shows this directly. Averaged over row 3 (columns 1–8), SARSA's value is -14.6 while Q-learning's is -5.35 , exactly V^* . (Q-learning matches V^* on rows 2–3 and at S; only the rarely visited rows 0–1 are still optimistic, by up to 2.4, because Q starts at 0.) SARSA values its top row (-11.5) above its edge row, so its greedy policy climbs. **highest_row is therefore a direct readout of how much each method's objective is affected by its own exploration.**

SARSA: $\max_a \bar{Q}(s, a)$ and $\operatorname{argmax}_a \bar{Q}(s, a)$



Q-learning: $\max_a \bar{Q}(s, a)$ and $\operatorname{argmax}_a \bar{Q}(s, a)$



Figure 3 [supp.]. Learned values $\max_a \bar{Q}(s, a)$ and greedy action $\operatorname{argmax}_a \bar{Q}(s, a)$ of the seed-averaged $\bar{Q}$ (seeds 0–4). SARSA's values drop sharply next to the cliff; Q-learning's equal V^* along the edge.

3 The effect of ϵ

We trained both algorithms with $\epsilon \in \{0.01, 0.05, 0.1, 0.2, 0.3\}$, everything else fixed, with 5 seeds (results_sweep/) and again with 20 seeds [supp.]. The conclusions agree; Figure 4 and Table 2 report the 20-seed runs, which is why their $\epsilon = 0.1$ row differs slightly from results.json. Besides training return and final greedy return we measured how many training episodes contain a cliff fall, and **greedy stability**: at the end of each of the last 500 episodes, does the current greedy policy reach G?

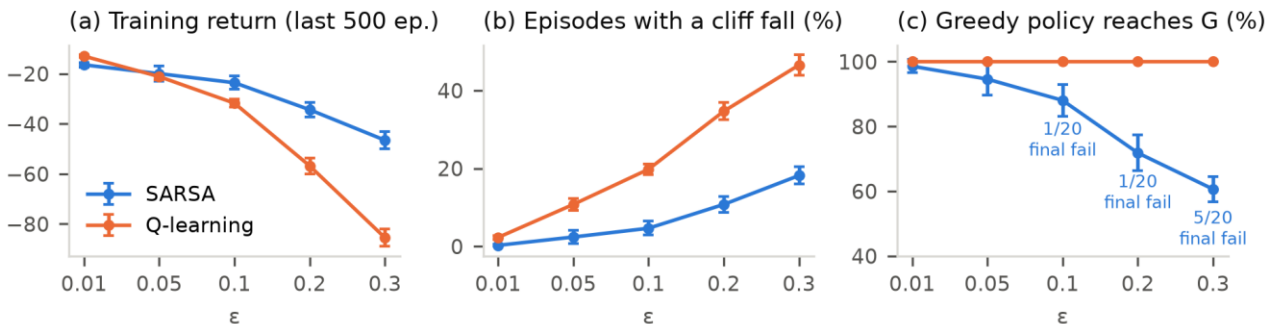


Figure 4 [supp.]. ϵ sweep, 20 seeds, mean ± 1 std across seeds; (a, b) over the last 500 training episodes. (c) Share of late-training greedy snapshots that reach G; the labels count runs whose final greedy policy never reaches G.

ϵ	Training return S / Q	Episodes with a fall S / Q	SARSA final greedy: reach G · return	SARSA late snapshots reaching G
0.01	-16.5 ± 0.9 / -13.0 ± 0.5	0.2 % / 2.3 %	20/20 · -16.1	98.6 %
0.05	-20.0 ± 3.0 / -21.2 ± 1.3	2.4 % / 10.7 %	20/20 · -16.3	94.6 %
0.1	-23.6 ± 2.7 / -31.8 ± 1.5	4.6 % / 19.8 %	19/20 · -15.8	88.0 %
0.2	-34.4 ± 2.9 / -56.8 ± 3.1	10.7 % / 34.6 %	19/20 · -16.9	71.8 %
0.3	-46.6 ± 3.4 / -85.5 ± 3.4	18.2 % / 46.5 %	15/20 · -16.9	60.6 %

Table 2 [supp.]. 20 seeds, mean $\pm$ std; S = SARSA, Q = Q-learning; bold = better training return. SARSA's final greedy return is averaged over the runs that reach G only (a failed run has no finite return), so it is conditional on success. Q-learning reaches G on 20/20 seeds with return -11 and in 100 % of snapshots at every ϵ .

Exploration and training return. Larger ϵ means more random moves and more falls, but the cost depends on where the agent walks. On the edge, Q-learning falls in 2.3 % $\rightarrow$ 46.5 % of episodes as ϵ goes from 0.01 to 0.3; SARSA, away from the edge, in 0.2 % $\rightarrow$ 18.2 %. Training return drops for both, much faster for Q-learning ($-13.0 \rightarrow -85.5$ vs $-16.5 \rightarrow -46.6$), and the ranking reverses: Q-learning trains better at $\epsilon = 0.01$, they are level at 0.05, and SARSA is better from 0.1 on.

Final greedy evaluation. Q-learning's greedy policy is optimal (-11) at every ϵ and seed, because its target does not depend on ϵ . SARSA's greedy path stays on the detour at every ϵ and becomes less reliable as ϵ grows: the share of late snapshots whose greedy policy reaches G falls from 98.6 % to 60.6 %, and at $\epsilon = 0.3$, 5 of 20 final policies never reach G.

These failures are not a bug. The spec notes that a greedy policy which fails to reach G signals a bug, because this cannot happen *at convergence*. Constant- α SARSA does not converge: its target samples a' from the exploring policy (a random a' near the cliff is worth about -100), and with $\alpha = 0.5$ each noisy target moves Q halfway, so near-tied actions keep swapping. A greedy policy can then contain a loop. For example, at S ($\epsilon = 0.2$, seed 3) $Q(\text{up}) = -24.02 < Q(\text{down}) = -23.85$, and *down* bumps the border, so the greedy agent never leaves S; the ϵ -greedy behaviour escapes such loops, so training is unaffected. Q-learning, whose target is deterministic here, never fails, and all five graded SARSA runs ($\epsilon = 0.1$, results.json) reach G.

Too little exploration also has a cost. At $\epsilon = 0.01$ the edge path is nearly the best choice even for SARSA's own objective: executed with $\epsilon = 0.01$, the edge path earns -12.8 against -17.0 for SARSA's detour (Figure 6). Yet SARSA does not find it. Even in a 20,000-episode run at $\epsilon = 0.01$ [supp.] its greedy path only drifts from row 0 to rows 1–2. Early episodes, with Q initialised at 0, fall off the cliff often, so edge actions start with very negative values; with $\epsilon = 0.01$ they are almost never retried, and those estimates are not corrected. In short, **ϵ trades training cost against discovery and, for SARSA, against greedy reliability.**

4 Worse training return, better greedy return

Both facts hold because the two numbers measure different policies. At the spec settings, Q-learning's training return is worse (-31.7 vs -23.2) while its greedy return is better (-11.0 vs -16.6). Training return measures the ϵ -greedy *behaviour* policy while it learns; greedy return measures the learned policy with exploration switched off (Figure 5).

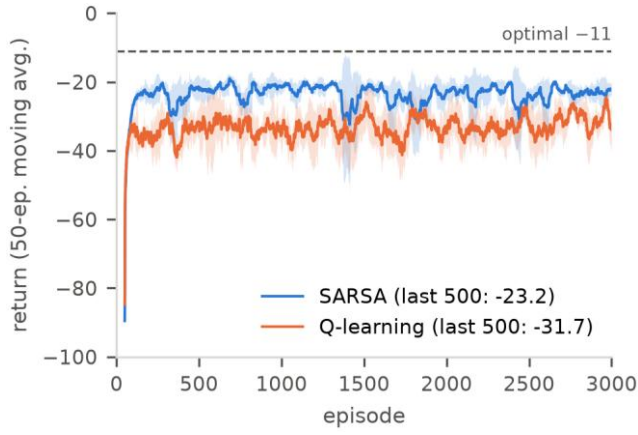


Figure 5. Training return at the spec settings ($\epsilon = 0.1$), mean ± 1 std over seeds 0–4, 50-episode moving average, y-axis clipped at -100. Legend: raw last-500 means, as in results.json.

To separate the two, we froze each learned Q and executed its policy without learning at different ϵ [supp.]. At $\epsilon = 0.1$, SARSA's policy earns -20.9 and Q-learning's -31.7, the same as the DP-optimal policy executed ϵ -greedily (-31.8). At $\epsilon = 0$ the order flips. The curves cross near $\epsilon \approx 0.04$ (Table 3, Figure 6).

Seeds 0–4	SARSA	Q-learning
Training return ($\epsilon = 0.1$, learning)	-23.2 $\pm$ 1.8	-31.7 $\pm$ 0.6
Frozen policy at $\epsilon = 0.1$ [supp.]	-20.9 $\pm$ 0.9	-31.7 $\pm$ 1.3
Cliff falls / episode at $\epsilon = 0.1$ [supp.]	0.03	0.24
Greedy return ($\epsilon = 0$)	-16.6 $\pm$ 0.8	-11.0 $\pm$ 0.0
highest_row	0.2	3.0

Table 3. Mean $\pm$ std over seeds; frozen-policy rows average 2,000 episodes per seed.

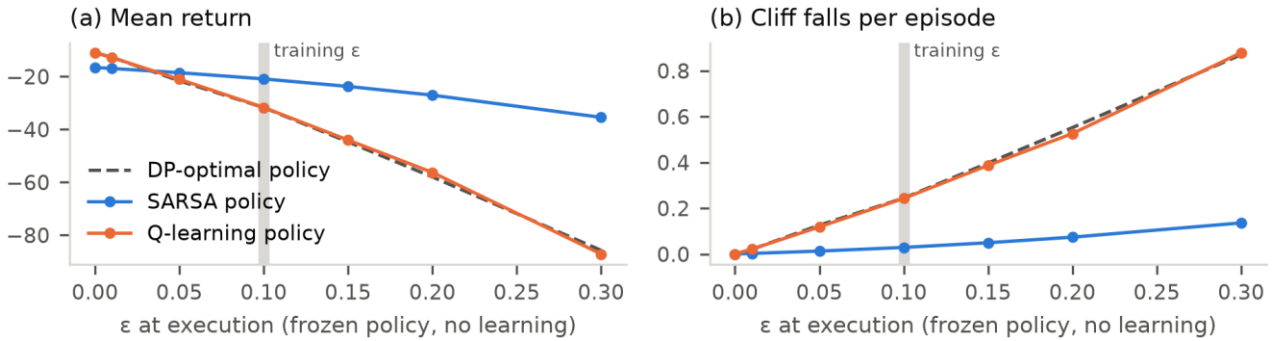


Figure 6 [supp.]. Policies learned at $\epsilon = 0.1$, frozen and executed at different ϵ . Q-learning's policy behaves exactly like the DP-optimal one (dashed).

What each algorithm is estimating. Q-learning is off-policy: it estimates q^* , the value of acting optimally with no future random moves. Its greedy policy is therefore optimal once exploration stops, but during training that policy is executed ϵ -greedily on the most dangerous route (0.24 falls per episode). SARSA is on-policy: it estimates q_π of the ϵ -greedy policy it actually runs, random moves included, so it tracks the policy that is best *given that it keeps exploring*: a safer route with fewer falls and a better training return, but not the greedy optimum. Neither is wrong; the right choice depends on whether exploration continues when the policy is used. Decaying ϵ linearly from 0.1 to 0 [supp.] confirms this: Q-learning's training return then reaches -11, equal to its greedy return, while SARSA's training and greedy returns both settle at -16.3 on the detour. With constant α and a finite budget, SARSA's early pessimism about the edge is locked in; reaching the optimum requires GLIE exploration *and* decreasing step sizes (Singh et al., 2000).

5 Conclusions

- **DP:** PI and VI give identical V^* and policies. VI needs 12 sweeps; PI needs 9 rounds but 3,276 evaluation sweeps, because early policies loop and (cold-started) evaluation converges only at rate γ .
- **On- vs off-policy:** Q-learning learns q^* and the edge path (highest_row 3.0, return -11). SARSA learns the value of its ϵ -greedy behaviour, prices in the 2.5 % per-step fall risk and detours (highest_row 0.2, return -16.6).
- **ϵ :** higher ϵ raises falls and lowers training return, far more for Q-learning, and makes SARSA's greedy policy less reliable (98.6 % $\rightarrow$ 60.6 %). Very small ϵ reverses the training-return ranking and starves SARSA of the exploration it needs to revise early estimates.
- **Training vs greedy:** SARSA wins while exploring (-20.9 vs -31.7 at $\epsilon = 0.1$), Q-learning wins without exploration (-11.0 vs -16.6); the crossover is near $\epsilon \approx 0.04$.

Limitations. Graded results use 5 seeds (supplementary runs use 20). α is constant, so SARSA does not converge and its final greedy policy is a snapshot of a moving estimate. All findings come from one small deterministic environment. Further supplementary figures (PI/VI convergence trace, learning curves per ϵ , decaying- ϵ and 20,000-episode runs) are in figures/extra_*.png.

References. R. S. Sutton & A. G. Barto (2018), *Reinforcement Learning: An Introduction*, 2nd ed., Example 6.6. S. Singh, T. Jaakkola, M. L. Littman & C. Szepesvári (2000), Convergence results for single-step on-policy RL algorithms, *Machine Learning* 38:287–308.